Pacgate AI - Two-AIPC Deployment Handbook

> Clone the repo on each machine, run the same install steps, and both machines become fully operational with deer-flow research and qm collaboration. Targets release 0.1.28 - handbook updated 2026-10-13 Chinese version: AIPC-DEPLOYMENT-HANDBOOK-ZH.md Prerequisites: Docker Desktop, Ollama, Node.js 24+. `install.ps1` pulls the models listed in `ollama-models.txt`. PowerShell 7 (`pwsh`) is optional - the installer runs on the built-in Windows PowerShell 5.1 as well. (Before 2026-10-11 that claim was false, in two ways: PS 5.1 turns *any* native command that writes to stderr into a terminating error when a redirection is attached, and `docker compose up -d` writes progress to stderr on every run, so the installer aborted before bootstrapping the tenant and admin; and the admin-registration call used the PowerShell 7-only `-SkipHttpErrorCheck` parameter, which 5.1 rejects before sending the request. `install.ps1` , `setup-qm.ps1` and `smoke-deer-flow-outputs.ps1` now route every stderr-writing call through `Invoke-WithNativeStderr` and make API calls through `Invoke-ApiRequest` . See `deploy/DEFECT-ps51-native-stderr.md` .)

⚠️ Significant findings (2026-09-02) — read before deploying AIPC #2

These were discovered during the AIPC #1 pilot and are **already fixed in this repo**. AIPC #2 must pull the **updated** code (see Stage 1) so it gets these fixes.

> **Update 2026-09-23 (supersedes the 2026-09-15 "clone either" note).** The two repos are **NO LONGER identical** — `pacgate-ai/pacgate-ai-pr` is 26 commits behind and missing the workflow-wiring fixes (`b7fc540` , `039afdc`), so a fork clone serves **10 built-in workflows instead of the firm's 222**, with no error shown anywhere. **Clone `JZKK720/pacgate-ai-pr`** (see Stage 1). Which repo you push a release tag to still decides the GHCR namespace — see `plans/012-master-release-namespace.md` .

1. **deer-flow agent could not query pacgate's legal databases.** Root cause: no tool was wired to pacgate-api's `/api/kb/search` (RAG) or `/api/search` (legal connectors), and the memory adapter silently fell back to local files (`PacgateMemoryStorage requires PACGATE_MATTER_ID`). **Fix:** a new `pacgate-mcp` service (FastMCP) exposes `pacgate_kb_search` , `pacgate_connector_search` , `pacgate_list_connectors` to deer-flow. Registered in `deer-flow-extensions-config.json` beside openviking.
2. **openviking MCP had the wrong API key baked in.** `deer-flow-extensions-config.json` used the app key (`OPENVIKING_API_KEY`) but openviking's `root_api_key` is `OPENVIKING_ROOT_API_KEY` . A

wrong key made openviking return 401, which rolled back the **entire** MCP tool load (deer-flow uses `asyncio.gather`), so **no** MCP tools appeared. **Fix:** use `OPENVIKING_ROOT_API_KEY` (template now uses `${OPENVIKING_ROOT_API_KEY}`).

3. `docker compose up -d --force-recreate deer-flow` **wipes deer-flow's local DB.** The SQLite DB, admin user, threads, and `.jwt_secret` live at `/app/backend/.deer-flow/` **inside the container** (not mounted). Recreating the container loses them → the frontend gets 401 and the `/setup` page appears. **Use** `docker compose restart deer-flow` for config changes; only recreate if you accept losing the local DB (then re-run `/setup`).

4. **QM sign-in needs an email transport, not Outlook SMTP.** The old SMTP path (`smtp.office365.com` + app password) is broken — Microsoft retired Basic Auth / app passwords for Exchange Online (Sep 2025). `qm check` failed with `535 5.7.139 Authentication unsuccessful`. **Fix:** qm's auth broker now supports two transports — **Mailpit SMTP catcher** (pilot; links land at `http://localhost:8025`, no key needed) and **Resend** (production; supply `RESEND_API_KEY`). See Stage 4. *(Originally this item said Resend was the only option; Stage 4's two-transport setup supersedes it.)*

5. **The qm web-ui cannot self-authenticate.** Its server (`/app/server/index.ts`) sets `AUTH_MODE = COOKIE_AUTH ? "dev" : "portal"`. Because `CORE_SIGNING_SECRET` is set, it's in **portal** mode and requires a portal-issued identity token. There is **no no-secret way** to reach the web-ui directly — you must run `portal` + `auth` (Resend) or an external OIDC provider. `ADMIN_GRANTS` is an authorization seed, not a sign-in.

6. **Git push to** `JZKK720/pacgate-ai-pr` **was blocked for the** `pacgate-ai` **account** (403, needs 2FA grant). **Workaround:** the `pacgate-ai` account could fork and push there. All fixes are now merged into both `main` branches, so this is historical — but note the two remotes are separate publishing targets for GHCR (see `plans/012-master-release-namespace.md`).

Architecture: two identical machines

Both AIPCs run the complete stack:

```
Each AIPC machine:
  nginx :8089  → pacgate-api :8080 (Rust metadata API)
               → deer-flow  :8001 (research workspace)
  Postgres :5432 (local metadata DB)
  OpenViking :1933 (long-term memory lane, MCP)
  qm :8182 (co-working workspace, runs via `qm up`)
  Ollama :11434 (native, GPU/NPU)
```

Each machine is self-contained and independently operational. Lawyers on either machine can use both research mode (deer-flow at `http://localhost:8089/research/`) and collaboration mode (qm at `http://localhost:8182`) without depending on the other machine.

If you later want shared matter data across both machines, connect them with a private mesh (Tailscale or WireGuard) and decide on a sync or single-authority model. That is a post-pilot decision, not a deployment prerequisite.

What you need before starting

- GitHub access to the source repo — `JZKK720/pacgate-ai-pr` (public; a plain clone needs no auth). **Do not clone the `pacgate-ai` fork** — it is 26 commits behind and serves 10 workflows instead of 222 (see Stage 1). A PAT or `gh auth login` is only required if you intend to push.
- Docker Desktop running on both AIPCs
- Ollama running on both AIPCs (`install.ps1` pulls the models it needs)
- `ollama signin` completed on each AIPC if the cloud-tagged deepseek models are in use
- Node.js 24+ installed on both AIPCs (for qm)
- **No `docker login ghcr.io` needed** — the Pacgate runtime images are published as **public** GHCR packages (see Stage 0).

Stage 0: Runtime images (dev machine, already done)

The runtime is published on GHCR and needs no rebuild on the AIPC.

Current versions (the ones to pull):

Image	Status
`ghcr.io/jzkk720/pacgate-api:0.1.30`	Published, public. Adds the matter-workspace rollup (`GET /api/matters/:id/workspace`).
`ghcr.io/jzkk720/pacgate-mcp:0.1.30`	Published, public. Exposes 19 MCP tools to deer-flow (adds `pacgate_get_workspace` , `pacgate_read_memory` , `pacgate_write_memory`).
`ghcr.io/jzkk720/deer-flow-pacgate:0.1.30`	Published, public.
`ghcr.io/jzkk720/deer-flow-frontend-pacgate:0.1.30`	Published, public.
`ghcr.io/jzkk720/ocr-service:0.1.30`	Published, public. PaddleOCR extraction; first-class since 0.1.16. Its PP-OCRv4 weights are **baked into the image** from 0.1.28, so extraction needs no external model host and works offline.
`ghcr.io/volcengine/openviking@sha256:46f9e34c…`	Pinned by digest in `compose.prod.yaml` . Upstream public image.

> Namespace and version corrected 2026-09-22. This table previously listed `ghcr.io/pacgate-ai/*` at 0.1.0/0.1.3. `pacgate-ai` is the legacy mirror; the live namespace is `jzkk720` and publishing moved there in plan 016. The old `pacgate-ai/ … -frontend-pacgate:0.1.0` row was also labelled "Published" while returning **404** - it never resolved. All five `jzkk720/*` images at the current pin (0.1.28, table above) return HTTP 200 anonymously.

Historical release table (retained for provenance, superseded):

Image	Status
`ghcr.io/pacgate-ai/pacgate-api:0.1.3`	Published at the time. Fixed the 0.1.1 container-networking bug.
`ghcr.io/pacgate-ai/pacgate-mcp:0.1.3`	Published at the time.
`ghcr.io/pacgate-ai/deer-flow-pacgate:0.1.3`	Published at the time.
`ghcr.io/pacgate-ai/deer-flow-frontend-pacgate:0.1.0`	**Never published - 404.** Do not use.

All Pacgate packages must be set to public visibility on GHCR so an AIPC can pull without registry credentials. Verify before rollout:

```
# Expect HTTP 200 with no docker login. 401/403 means the package is still private.
# 404 means the tag does not exist in that namespace - usually a drift between the
# pins in compose.prod.yaml and where the images were actually published.
#
# (The Accept header is REQUIRED - omit it and a public manifest returns 404, not 200.)
#
# This reads the four pins straight out of compose.prod.yaml, so it can never go
# stale: previously this snippet hard-coded an old tag that had been removed, and
# reported a healthy system as broken.
$acc = "application/vnd.oci.image.index.v1+json,application/vnd.docker.distribution.manifest.
$compose = "deploy/client-bundle/compose.prod.yaml"
$pins = Select-String -Path $compose -Pattern "image:\s*(ghcr\.io/[^\s]+)" -AllMatches |
        ForEach-Object { $_.Matches } | ForEach-Object { $_.Groups[1].Value } |
        Where-Object { $_ -notmatch "openviking" } | Sort-Object -Unique

foreach ($pin in $pins) {
  $repo = $pin -replace "^ghcr\.io/", ""            # owner/name:tag
  $name = ($repo -split ":")[0]                       # owner/name
  $tag  = ($repo -split ":")[1]
  $t = (Invoke-RestMethod "https://ghcr.io/token?scope=repository:$name`:pull").token
  $code = (Invoke-WebRequest -Uri "https://ghcr.io/v2/$name/manifests/$tag" `
            -Headers @{Authorization="Bearer $t"; Accept=$acc} `
            -Method Head -UseBasicParsing).StatusCode
  "{0,-58} => HTTP {1}" -f $pin, $code
}
```

To flip it (GitHub web UI — the API route 404s for personal accounts): GitHub → your profile → Packages → `pacgate-api` → Package settings → Visibility → **Public** → Save. Repeat for `deer-flow-pacgate`. This is safe: the images contain only the compiled binary and SQL migrations, every secret is injected at runtime via `.env`, and the installer already has full source access to the same code.

Only rebuild and push if the Rust source changes, from the dev machine. **Take the tag from the compose pin rather than typing a version** — a hard-coded tag here has gone stale twice, and one stale copy already reported a healthy system as broken:

```
cd c:\Users\cubecloud-io\github-pr\pacgate-ai-pr
# Pattern reads the CURRENT namespace (jzkk720). The old form matched
# 'pacgate-ai/pacgate-api' and no longer matches any line in compose.prod.yaml,
# so $tag came back EMPTY and the build/push below silently used a blank tag.
$tag = (Select-String -Path deploy/client-bundle/compose.prod.yaml `
        -Pattern 'jzkk720/pacgate-api:(\S+)').Matches.Groups[1].Value
if (-not $tag) { throw 'could not read the image tag from compose.prod.yaml' }
docker build -t ghcr.io/jzkk720/pacgate-api:$tag -f pacgate-ai/Dockerfile ./pacgate-ai
docker push  ghcr.io/jzkk720/pacgate-api:$tag
```

In practice prefer the `build-ghcr.yml` workflow so all four images stay in step — see `deploy/README-BUILD.md` and `plans/012-master-release-namespace.md`.

Do **not** rebuild on the AIPC — the pilot runs the published digests.

> **Port note:** the stack binds nginx to host port `8089` (the value committed in `deploy/client-bundle/compose.prod.yaml`). If that port is already in use on the machine, edit the `ports:` entry for `nginx` and use the new port in all verification URLs below.

Stage 1: Clone the repo on each AIPC

On both machines:

```
cd C:\
git clone https://github.com/JZKK720/pacgate-ai-pr.git
cd pacgate-ai-pr
git remote -v   # origin MUST be JZKK720/pacgate-ai-pr
```

> **AIPC #2 note (CORRECTED 2026-09-23):** the two repos are **NO LONGER IDENTICAL**, so "either clone works" is no longer true. **Clone `JZKK720`.**
>
> `pacgate-ai/pacgate-ai-pr` is **26 commits behind** (last checked 2026-09-23) and is missing `b7fc540` and `039afdc`, so it still carries the **original workflow-wiring defect**. The 15 workflow YAMLs are present in the fork but are not wired into `pacgate-api`, so the API serves **10 built-in workflows instead of the firm's 222** — with no error shown anywhere. The identity claim above was accurate when written; the divergence came afterwards, which is exactly why a doc must not assert two things are in sync without a check that can fail.
>
> The namespace difference is unchanged — which repo you push a release tag to decides which GHCR namespace the images publish into. See `plans/012-master-release-namespace.md`.
>
> Cloning needs no credentials now that the repos are public; a PAT or `gh auth login` is only required to push.

If the repo is private and GitHub prompts for credentials, use a personal access token or the GitHub CLI (`gh auth login`).

Stage 2: Deploy the core stack (both machines, identical steps)

Run these steps on each AIPC. The Docker Compose stack starts pacgate-api, Postgres, nginx, and deer-flow.

```
cd C:\pacgate-ai-pr\deploy\client-bundle
copy .env.example .env
notepad .env
```

Fill in these values:

```
PACGATE_DB_PASSWORD=<generate a strong password>
PACGATE_JWT_SECRET=<generate a random hex string>
PACGATE_TENANT_ID=pacgate-law
OPENVIKING_ROOT_API_KEY=<generate a 32-char hex string>
OPENVIKING_API_KEY=<generate a 32-char hex string>
```

`OPENVIKING_API_KEY` is **required** — the installer renders `deer-flow-extensions-config.json` from it and stops with an error if it is missing or left as `change-me`.

Generate secrets if you need them:

```
# DB password (16 hex)
-join ((1..16) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# JWT secret (32 hex)
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# OpenViking keys (32 hex each) — generate a fresh value per line
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })
```

Run the installer:

```
.\install.ps1
```

The installer pulls the (public, no-login) GHCR images, renders `OPENVIKING_CONF_CONTENT` and `deer-flow-extensions-config.json` from the `.env` secrets, starts the Docker Compose stack, and pulls the Ollama models listed in `ollama-models.txt`. If models are already pulled, this step is fast.

Verify the core stack:

```
docker compose -f compose.prod.yaml ps
curl http://localhost:8089/version
curl http://localhost:8089/pacgate/health
```

Expected: all eight containers running (pacgate-db, pacgate-api, deer-flow, deer-flow-frontend, pacgate-mcp, ocr-service, openviking, nginx); `/version` returns `{"version":"0.1.28","revision":"<git sha>"}` and `/pacgate/health` returns `ok`.

> **Do not probe `/health` at the nginx root.** nginx routes `/` to the deer-flow frontend by design, so `curl http://localhost:8089/health` returns the frontend's 404 page - that reads as a failure but is correct routing. `/version` is also at the root (mapped to the API's `/build-info`); only `/pacgate/*` paths reach the API.

Stage 3: Seed the tenant and provision accounts (both machines)

> **`install.ps1` now does this automatically (step 6a).** On a current install you do not need to run anything on this page - it is kept for recovery, and the commands below are what the installer runs. Check the install output for `[OK] tenant 'default-firm' present` and `[OK] admin '...' registered`.

How accounts work since 0.1.22 (read before provisioning people)

- `POST /api/auth/register` is **first-user-only**: it creates exactly one account on a fresh deployment (the bootstrap admin) and refuses afterwards with 403.
- Every account after the first is created by the admin through `POST /api/auth/users` (Bearer = the admin's token).
- Research workspace sign-in = email + password. Collaboration (qm) sign-in = one-time emailed links allowlisted via `AUTH_ALLOWED_EMAILS`.

On each machine, seed the default tenant, then let the installer bootstrap the admin:

```
# Seed the tenant (idempotent).
#
# THE SLUG MUST MATCH PACGATE_TENANT_ID. It defaults to "default-firm", and the
# registration below looks the tenant up by that slug. An earlier version of this
# page used 'pacgate-law', which does not match, so registration failed with:
#
#   {"error":{"code":"internal_error",
#     "message":"default tenant not found: matter not found: row not found"}}
#
# That reads like a database fault and is really a naming mismatch. If you set
# PACGATE_TENANT_ID to something else in .env, use that value here instead.
docker exec pacgate-db psql -U pacgate -d pacgate -c "INSERT INTO tenants (name, slug) SELECT
```

Provision an attorney user (run by the admin once the admin account exists):

```
$login = Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/login" -Method Post `
  -Body '{"email":"admin@pacgate-law.com","password":"<admin-password>"}' `
  -ContentType "application/json"
$body = @{email="<attorney-email>"; password="<attorney-password>"; role="attorney"} | Conver
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method Post `
  -Headers @{Authorization="Bearer $($login.token)"} -Body $body -ContentType "application/js
```

Register the qm bridge service account (the installer does this; manual for recovery). Same admin-provisioned route, same shape as above, with `email="qm-bridge@pacgate-law.com"`.

For the qm collaboration surface: set `AUTH_ALLOWED_EMAILS` (comma-separated) and `ADMIN_GRANTS=<email>:org_admin` in `deploy/client-bundle/qm-pacgate/.env`, then re-run `setup-qm.ps1`.

Stage 3.5: Verify OpenViking memory service (both machines)

OpenViking is the long-term memory lane: deer-flow and qm store conversational context there and recall it in later sessions. It starts as part of the compose stack.

```
curl http://localhost:1933/health
```

Expected: `{"status":"ok","healthy":true, ...}`. The installer renders the OpenViking config (Ollama embedding + VLM) into `.env` as `OPENVIKING_CONF_CONTENT` and seeds the server's `ov.conf` on first boot.

Functional check (optional, uses the root key from `.env`):

```
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: applicat
```

Expected: a tool list including `find`, `search`, `read`, `remember`.

Boundary rule: OpenViking stores conversational context only (decisions, preferences, working knowledge). Matter documents and T1-T4-controlled content stay in pacgate-api/pacgate-rag.

Stage 4: Bootstrap qm (both machines, identical steps)

qm runs separately from the Docker Compose stack. Bootstrap it on each machine after the core stack is healthy.

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\setup-qm.ps1
```

The script prompts for: - Administrator work email (lowercased) - Pacgate bridge email: `qm-bridge@pacgate.local` - Pacgate bridge password: the one you registered in Stage 3

The script generates signing secrets, creates `.env` in the qm-pacgate directory, validates the config with `qm check`, builds the sandbox image with `qm sandbox build`, fetches the static docker CLI the core shells (SHA-256 verified), and builds the `qm-pacgate-sandbox-local` exec-daemon wrapper image.

QM sign-in needs an email transport. The qm auth broker delivers sign-in one-time links. Two supported transports:

- **Local/pilot topology (Mailpit SMTP catcher):** `SMTP_HOST=mailpit SMTP_PORT=1025 AUTH_EMAIL_TRANSPORT=smtp SMTP_TLS=none`. Links LAND IN THE MAILPIT INBOX — open `http://localhost:8025` to pick up a sign-in link in a pilot. No real email is sent.
- **Production topology (Resend):** `AUTH_EMAIL_TRANSPORT=resend` with `RESEND_API_KEY` in `deploy/qm-pacgate/.env`, and `AUTH_EMAIL_FROM` set to a Resend-verified sender (an Outlook address is not verified; Microsoft retired Basic Auth/app passwords for Exchange Online, so raw Outlook SMTP is NOT a supported transport):

```
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

Start qm:

```
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d
```

> **Use compose, NOT `qm up`.** The `@yc-software/qm` CLI's docker lifecycle is POSIX-only - its `which()` shells `/bin/sh`, which does not exist on Windows (measured 2026-10-06: `execFileSync("/bin/sh", …)` → `ENOENT`), so `qm up`, `qm down`, and `qm status` die with "docker not found on PATH" on every AIPC. `compose.qm.yaml` describes the same topology (same names, volumes, network) and additionally carries the Pacgate patches: `patch/pi-models.ts` and `patch/local-sandbox.ts` are bind-mounted over the core's source, the static docker CLI and the host docker socket are mounted, and the sandbox wrapper image is wired. Those mounts survive every `down` / `up -d` cycle - the model routing and the sandbox lane are durable through compose. Do NOT run `docker compose -f compose.qm.yaml up -d` and a `qm up` against the same directory (they would fight over the same named volumes and network).

Verify qm:

```
# Open http://localhost:8182  (web-ui) — requires a portal identity token
# Open http://localhost:8181  (portal) — the sign-in front door
# Sign in with the admin email (magic link via the configured transport; Mailpit in pilot)
# Send a test message
# Ask: "List available pacgate workflows"
```

> **Web-ui auth reality:** the qm web-ui (`:8182`) cannot authenticate anyone on its own — it must be reached through the portal (`:8181`), which issues the identity token. If you open `:8182` directly you'll see "reached through the portal." Always go through `:8181` .
>
> **Dev-mode alternative (pilot / single-user):** for a local pilot you can run qm in **dev/cookie mode** instead of the portal. Recreate `qm-pacgate-core` with `NODE_ENV=development` + `ALLOW_UNAUTHENTICATED_CORE=1` and **no** `CORE_SIGNING_SECRET` , and `qm-pacgate-web-ui` with **no** `CORE_SIGNING_SECRET` . Then `POST /signin` works directly at `:8182` with `{"user":"<principal>"}` and no Resend key is needed. This is **not** production-correct (no auth) — use it only for a single-user pilot.

Stage 5: Verify deer-flow (both machines)

On each machine, verify the research workspace:

```
# Open http://localhost:8089/research/
# Select or create a matter
# Ask: "Summarize recent force majeure case law in China"
# Verify: response includes citations
# Verify: response is saved to matter memory
```

Stage 5.5: Full-loop operations — deer-flow ↔ QM ↔ OpenViking

Both workspaces share **one OpenViking** long-term memory lane and **one pacgate-api** metadata store. This section documents how the two systems talk to each other and how to verify the loop end-to-end.

Topology (verified 2026-09-04)

```
pacgate-ai-bundle_default  (Docker Compose network)
├── openviking :1933   ← long-term memory (MCP: find/search/read/remember)
├── pacgate-api :8080  ← metadata API (matters/workflows/connectors)
├── pacgate-mcp :8000  ← FastMCP bridge exposing pacgate KB/connector search
├── deer-flow :8001    ← research workspace (consumes openviking + pacgate-mcp)
└── nginx :8089        ← ingress (deer-flow frontend + /pacgate/ API)

qm-pacgate  (qm up network)
├── qm-pacgate-core    ← co-working agent runtime
├── qm-pacgate-web-ui  ← browser chat UI (:8182)
└── qm-pacgate-pg      ← qm Postgres

Bridge: qm-pacgate-core is ALSO joined to pacgate-ai-bundle_default, so it can
reach openviking:1933, pacgate-api:8080, and host.docker.internal:11434 (Ollama).
```

How the loop works

1. **deer-flow → OpenViking**: `deer-flow-extensions-config.json` registers `openviking` as an HTTP MCP server at `http://openviking:1933/mcp` with the **root** API key (`OPENVIKING_ROOT_API_KEY`). Research runs store and recall conversational context there.
2. **deer-flow → pacgate**: `pacgate-mcp` (FastMCP) exposes `pacgate_kb_search` / `pacgate_connector_search` / `pacgate_list_connectors` to deer-flow, backed by pacgate-api's RAG + legal connectors.
3. **QM → OpenViking**: the QM core has `OPENVIKING_URL=http://openviking:1933` and `OPENVIKING_API_KEY` (the root key). The `pacgate-qm` sandbox tool (`ov-remember` / `ov-search` / `ov-read`) calls OpenViking from inside the agent sandbox at `http://host.docker.internal:1933` with `OPENVIKING_ACCOUNT=pacgate-law` + `OPENVIKING_USER` .
4. **QM → pacgate**: the `pacgate-qm` sandbox tool logs into pacgate-api (`PACGATE_API_EMAIL` / `PACGATE_API_PASSWORD`) to discover workflows, bind a QM scope to a matter, read/write matter memory, and execute workflows.

Verify the full loop

```
# 1. OpenViking MCP responds (root key from .env)
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: applicat
# Expected: tools find/search/read/remember

# 2. QM core reaches openviking + pacgate-api + Ollama
docker exec qm-pacgate-core sh -c "curl -s -o /dev/null -w 'openviking:%{http_code}\n' http://

# 3. QM sign-in works (dev/cookie mode)
$body = '{"user":"admin@pacgate-law.com"}'
curl.exe -s -X POST http://localhost:8182/signin -H "Content-Type: application/json" -d $body
# Expected: {"ok":true,"user":"admin@pacgate-law.com"}
```

QM ↔ bundle network join (idempotent)

QM core is joined to the bundle network so it can resolve `openviking` and `pacgate-api` by name. If a `qm up` / `qm down` cycle drops the join, re-apply it:

```
docker network connect pacgate-ai-bundle_default qm-pacgate-core
```

> **Note:** QM is intentionally **not** a Docker Compose service in the bundle. It is managed by the `@yc-software/qm` CLI (`qm up` / `qm down`) with its own lifecycle. The network join is the only coupling — keep it that way. Do not rewrite QM as compose services; that fights the qm CLI and would recreate the containers on every `qm up` .

Stage 6: Smoke test checklist (both machines)

Run this checklist on each AIPC independently.

Core stack

- [ ] `docker compose -f compose.prod.yaml ps` shows 5 services up (incl. openviking)
- [ ] `curl http://localhost:8089/version` returns the release version + revision JSON
- [ ] `curl http://localhost:8089/pacgate/health` returns `ok` (NOT `/health` at the root - see Stage 2)
- [ ] `curl http://localhost:1933/health` returns healthy JSON
- [ ] Postgres has the `pacgate-law` tenant
- [ ] Admin user can log in at `http://localhost:8089/api/auth/login`

- [ ] deer-flow returns a real research response at `http://localhost:8089/research/`

qm collaboration

- [ ] `npm exec qm -- status` shows qm running
- [ ] `http://localhost:8182` loads the qm web UI
- [ ] Admin can sign in
- [ ] qm can list Pacgate workflow categories
- [ ] qm can execute one Pacgate workflow through the bridge

Ollama

- [ ] `ollama list` shows the required models
- [ ] deer-flow can call Ollama for inference
- [ ] qm can call Ollama for inference

Data

- [ ] `./data/tenants/` directory exists and is writable
- [ ] `./openviking/` directory exists and persists across restarts
- [ ] Document upload works through the API
- [ ] Matter memory persists after a deer-flow research run
- [ ] Cross-session recall: a fact stored via OpenViking `remember` is recalled via `search` in a later session

Managing the stack after deployment

Start and stop

```
# Start core stack
docker compose -f compose.prod.yaml up -d

# Stop core stack
docker compose -f compose.prod.yaml down

# Start qm
cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate
docker compose -f compose.qm.yaml up -d

# Stop qm (NO -v: .env and the volumes stay)
docker compose -f compose.qm.yaml down
```

> `qm up` / `qm down` cannot run on Windows (the CLI's `which()` shells `/bin/sh` → ENOENT); compose is the only working qm lifecycle on the AIPC and it carries the patches that keep model routing and the sandbox lane durable.

Update to a new version

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\install.ps1 -Update
```

`-Update` now does all of the following, so a separate `git pull` is no longer needed (it was the most easily forgotten step, and forgetting it silently ran new images against old config):

Step	What it does
1	Refreshes the repo working tree (fast-forward only)
2	Re-renders the MCP config from the template, backing up any change
3	Pulls new GHCR images
4	Restarts the stack, and reloads nginx

It refuses, rather than guessing, when the machine has local work:

- uncommitted changes in the repo -> the repo update is skipped, and the changed files are listed. Nothing is stashed, reset, or discarded.
- local commits the remote does not have -> it will not merge or rebase. The repo stays as-is and the rest of the update continues.

Use `-SkipRepoPull` if you deliberately want an image-only update:

```
.\install.ps1 -Update -SkipRepoPull
```

qm is reported, not auto-updated

If qm is running on the machine, `-Update` also checks whether its sandbox image still matches its source:

```
[OK] qm sandbox matches its source (38e062ec7c5c ... )
```

or

```
[WARN] qm sandbox source has CHANGED since the image was pinned.
       qm is running OLD skills and tools. Rebuild + repin:
         cd deploy\qm-pacgate
         npm exec qm -- sandbox build   # then repin the printed digest
         pwsh -File ..\..\scripts\qm-sandbox-fingerprint.ps1 -Write
```

This matters because qm's agent executes inside a sandbox image **pinned by digest** in `deploy/qm-pacgate/qm.config.jsonc`. Digest pinning is correct - the isolation boundary should be immutable - but it means a repo update can change `deploy/qm-pacgate/sandbox/` while the image stays exactly as it was. The agent then keeps running the old skills and tools, with no error anywhere.

`-Update` **reports** this rather than rebuilding on its own, because the rebuild needs Node 24 + npm + buildx and the digest must be repinned afterwards - a config change we should not make unattended on a client machine. A wrong automatic rebuild is a worse failure than a visible warning.

Data is preserved across an update: - `./data/tenants/` (volume mount) - matters, documents, memory - Postgres data (named volume) - metadata database

Is this machine current? Ask it.

```
curl.exe -s http://localhost:8089/version
```

```
{"version":"<release>","revision":"<git sha>"}
```

This reports the version **compiled into the running pacgate-api binary**, and the commit it was built from. It deliberately does not echo the compose pin or the image tag: those record what was *deployed*, and the failure worth catching is exactly the case where the deployed artifact and the running process disagree.

Before this existed there was no way to tell a current machine from one behind -both looked identical from the outside, and the only check was to SSH in and read the compose file, hoping the containers matched it.

Switch models

deer-flow (research workspace): 1. Edit `deer-flow-config.yaml` - reorder the `models` list (first entry = default) 2. Restart: `docker compose -f compose.prod.yaml restart deer-flow`

qm (co-working workspace): 1. Edit `deploy/client-bundle/qm-pacgate/qm.config.jsonc` if the model set changes 2. Recreate core (compose keeps the patch mounts, so the routing stays durable): `powershell cd C:\pacgate-ai-pr\deploy\client-bundle\qm-pacgate docker compose -f compose.qm.yaml up -d --force-recreate core`

Register new users

`POST /api/auth/register` is **first-user-only** and returns 403 after the bootstrap account (see Stage 3). Create every later account through the admin route:

```
# /pacgate prefix required - see the note in Stage 3. Bearer = admin's token
# from POST /pacgate/api/auth/login (see Stage 3 for the login step).
$body = @{email="<user>@pacgate-law.com"; password="<password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/pacgate/api/auth/users" -Method POST `
  -Headers @{Authorization="Bearer <admin-token>"} -Body $body -ContentType "application/json
```

Backup the database

```
docker exec pacgate-db pg_dump -U pacgate pacgate > backup.sql
```

Check logs

```
docker compose -f compose.prod.yaml logs -f pacgate-api
docker compose -f compose.prod.yaml logs -f deer-flow
```

Known limitations

- `qm up` / `qm down` **do not work on Windows.** The qm CLI's docker lifecycle is POSIX-only: its `which()` runs `execFileSync("/bin/sh", …)`, which is `ENOENT` on every Windows AIPC (measured 2026-10-06). Use the compose path above (`docker compose -f compose.qm.yaml up -d`) - same topology, same container names, and it carries the Pacgate patches, so nothing is lost across recreates.
- Each machine has its own independent Postgres and `./data/tenants/` directory. Matter data is not shared between machines unless you later add a private mesh and a sync or single-authority model.

- The PkuLaw connector token is expired. Regenerate it at `https://mcp.pkulaw.com` and set `PKULAW_API_KEY` in `.env` if China-law search is needed during the pilot.
- Four WASM crates (citation-check, clause-parser, doc-validator, rule-engine) remain stubs. These are future-blueprint work and do not affect Phase 1 pilot functionality.
- **Model selection:** the API defaults to models that may not exist on the target machine. After Stage 3, apply per-tenant model overrides so the LLM tiers point at models actually present in `ollama list` on that machine. Recommended pilot set (benchmarked 2026-08-28): `gemma4:12b-it-qat` (Main — 13s/tool-round, schema-valid tool calls, verified end-to-end), `qwen3.8:27b-mtp-q4_K_M` (Mid — 73s/tool-round, stronger quality for batch tabular review), `nomic-embed-text:latest` (embeddings). Avoid reasoning-mode models (e.g. nemotron) for interactive tiers — they can hang long docx generations. See `plans/007-aipc-full-installation-handoff.md` Appendix A for the SQL template.

Files referenced

File	Purpose
`deploy/client-bundle/compose.prod.yaml`	Docker Compose for pacgate-api + deer-flow + Postgres + nginx
`deploy/client-bundle/install.ps1`	One-click Windows installer for the core stack
`deploy/client-bundle/setup-qm.ps1`	qm bootstrap script (secrets, config, sandbox build)
`deploy/client-bundle/.env.example`	Template for client secrets
`deploy/client-bundle/ollama-models.txt`	Models to pre-pull
`deploy/client-bundle/deer-flow-config.yaml`	Multi-model deer-flow config (5 models, switchable)
`deploy/qm-pacgate/qm.config.jsonc`	qm local deployment config
`deploy/SETUP-AND-OPERATIONS.md`	Full 3-day on-site install guide (reference)
`deploy/DEPLOYMENT-GUIDE.md`	Engineer-level deployment details (reference)